UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ESTRUCTURA DE DATOS
PROYECTO 1 - FASE 1
PRIMER SEMESTRE 2026
Andy Jefferson González Fuentes 202231338

MANUAL TÉCNICO

Arquitectura del sistema

Capa	Clases	Responsabilidad
Entidad	Product	Estructura de datos del dominio
Estructuras	LinkedList, SortedLinkedList, AVLTree, BTree, BPlusTree	Indexado y búsqueda
Servicios	Logger, CSVLoader	I/O y validaciones
Control	Catalog	Coordinación e inserción atómica
Interfaz	Menu, Benchmark, main	Interacción con el usuario

La clase Catalog contiene toda operación de inserción, búsqueda o eliminación, que a su vez delega a cada estructura y garantiza la consistencia entre ellas.

Product

Estructura central del sistema. Todos los punteros en las estructuras apuntan a instancias de Product.

Campo	Tipo	Descripción	Clave en
name	string	Nombre del producto	AVL, SortedList
barcode	string	Codigo de barra (único)	LinkedList, duplicados
category	string	Categoría del producto	BPlusTree
expiry_date	string	Fecha YYYY-MM-DD	BTree
brand	string	Marca del producto	-
price	double	Precio (> 0)	-
stock	int	Unidades disponibles (>= 0)	-

Operator **==** compara por barcode (clave única).

Operator **<** compara por name (para el ordenamiento en AVL y SortedLinkedList).

Estructuras de datos (DISEÑO Y JUSTIFICACIONES BIG O)

1. LinkedList - lista enlazada no ordenada: Inserción siempre al **frente** en O(1). Búsqueda secuencial nodo a nodo. Su propósito principal es servir de base para las pruebas comparativas del benchmark y proveer búsqueda por barcode.

Operación	Complejidad	Justificación
insertFront	O(1)	No requiere recorrido; re-enlaza sólo el head
searchSequential	O(n)	Recorre nodo a nodo hasta encontrar el barcode
searchByName	O(n)	Recorre nodo a nodo hasta encontrar el name
remove	O(n)	Búsqueda previa O(n) + re-enlace O(1)

2. SortedLinkedList - lista enlazada ordenada

Mantiene orden alfabético por nombre en todo momento. La búsqueda usa **early stop**: si el nodo actual supera **lexicográficamente** la clave buscada, el elemento no existe y se detiene sin recorrer el resto.

Optimización de carga masiva: durante la carga de CSV se usa insertFront O(1) para todos los productos, seguido de un merge sort único al final en O(n log n), reduciendo el costo total de O(n^2) a O(n log n).

Operación	Complejidad	Justificación
insertSorted	O(n)	Búsqueda de posición correcta antes de enlazar
insertFront	O(1)	Solo para carga masiva; requiere sortInPlace después
sortInPlace	O(n log n)	Merge sort sobre la lista enlazada
searchSequential	O(n)	Recorrido con early stop
remove	O(n)	Búsqueda con early stop + re-enlace

3. AVLTree - árbol AVL por nombre

Árbol binario de búsqueda que garantiza |factor_balance| <= 1 en cada nodo mediante rotaciones automáticas. La altura máxima es O(log n), lo que garantiza busqueda, insercion y eliminacion logaritmicas en todos los casos.

El factor de balance se calcula como altura(izq) - altura(der). Hay cuatro casos de desbalance que se corrigen con rotaciones:

Caso	Condición	Corrección
LL	balance > 1 y hijo izq pesado a la izq	Rotación simple derecha
LR	balance > 1 y hijo izq pesado a la der	Rotación doble izq-der

Caso	Condición	Corrección
RR	balance < -1 y hijo der pesado a la der	Rotación simple izquierda
RL	balance < -1 y hijo der pesado a la izq	Rotación doble der-izq

Operación	Complejidad	Justificación
insert	O(log n)	Descenso hasta hoja + rotaciones en el camino de retorno
search	O(log n)	Búsqueda binaria por nombre; altura garantizada O(log n)
remove	O(log n)	Sucesor inorder para dos hijos + rotaciones en retorno
inorder	O(n)	Recorrido izq-raiz-der visita cada nodo exactamente una vez

4. BTree - arbol B por fecha de caducidad

Árbol B de grado mínimo t=4. Cada nodo tiene entre t-1=3 y 2t-1=7 claves. La raíz puede tener entre 1 y 7 claves. Con n=1000 la altura máxima es 5 niveles.

Justificación de t=4: maximiza la reducción de altura antes de entrar en la zona de rendimientos decrecientes. De t=2 a t=4 la altura baja de 10 a 5 niveles para n=1000. De t=4 en adelante, cada incremento reduce la altura a menos de 1 nivel adicional.

La clave expiry_date en formato **YYYY-MM-DD** es comparada como string, lo que produce el orden cronológico correcto sin conversión a fecha.

Operación	Complejidad	Justificación
insert	O(log n)	Splits preventivos al descender; como mucho un split por nivel
search	O(log n)	Búsqueda lineal dentro del nodo + descenso; t constante
remove	O(log n)	3 casos: hoja directa, predecesor/sucesor, merge
rangeSearch	O(log n + k)	Descenso O(log n) + recorrido inorder de k resultados

Casos de eliminación:

- A1: nodo hoja - eliminación directa desplazando claves
- A2a: nodo interno, hijo izquierdo tiene >= t claves - reemplazar con predecesor
- A2b: nodo interno, hijo derecho tiene >= t claves - reemplazar con sucesor
- A2c: ambos hijos tienen t-1 claves - merge de los dos hijos
- B: clave no está en el nodo - garantizar >= t claves antes de descender (redistribución o merge)

Clase Catalog (inserción atómica)

Catalog coordina las cinco estructuras. Toda operación de escritura pasa por ella para garantizar que las cinco estructuras siempre tengan el mismo conjunto de productos.

addProduct (inserción atómica con rollback)

Paso	Operación	Flag de rollback
1	LinkedList::insertFront(p)	insertedList
2	SortedLinkedList::insertFront/Sorted(p)	insertedSorted
3	AVLTree::insert(p)	insertedAVL
4	BTree::insert(p)	insertedBTree
5	BPlusTree::insert(p)	insertedBPlus

Si alguna inserción lanza una excepción, rollback() elimina el producto de las estructuras que ya lo recibieron en orden inverso.

Carga masiva (beginBulkLoad / endBulkLoad)

Durante la carga de CSV, el flag _bulkLoading activa el modo masivo: addProduct usa insertFront O(1) en SortedLinkedList en lugar de insertSorted O(n). Al finalizar, endBulkLoad llama sortInPlace() una sola vez en O(n log n), reduciendo el costo total de O(n^2) a O(n log n).

CSVLoader (carga y validaciones)

El método **load()** lee el archivo línea por línea, activa el modo bulk, valida cada línea y llama **addProduct** para los productos válidos. Al finalizar llama **endBulkLoad()** y cierra el archivo.

Validación	Método	Acción si falla
Archivo abrible	ifstream::is_open	Retorna false; logea error
Exactamente 7 campos	splitLine	logMalformed + skip
Barcode no vacio	validateFields	logMalformed + skip
Fecha YYYY-MM-DD	isValidDate	logMalformed + skip
Precio numerico > 0	isDouble	logMalformed + skip
Stock entero >= 0	isInt	logMalformed + skip
Categoría/Marca vacias	validateFields	logMalformed + skip
Barcode único	addProduct	logDuplicate + skip

splitLine respeta comillas dobles en los campos: un campo como "Café, Molido" con coma interna se parsea correctamente porque la coma esta dentro de comillas.

Benchmark de rendimiento

El Benchmark mide tres tipos de operaciones de forma independiente con N=20 operaciones por repetición y M=5 repeticiones:

Sección	Estructuras	Casos medidos
Búsqueda	LinkedList, SortedList, AVL	hit_random, miss, hit_first, hit_last
Inserción	LinkedList, SortedList, AVL, BTree, B+	general (inserción + limpieza inmediata)
Eliminación	LinkedList, SortedList, AVL, BTree, B+	general (eliminación + reinsercion)

Para la sección de inserción se insertan variantes con barcodes únicos y se eliminan inmediatamente después de medir, para no acumular productos en el catálogo. Para la eliminación se toma el primer producto de la lista como sujeto y se reinserta después de cada serie.

Visualización Graphviz

Árbol	Formato de nodos	Color
AVL	shape=box; etiqueta: nombre + altura (h=N)	sin relleno
BTree	shape=record con puertos; patrón: <p0>|<f0> fecha|<p1>...	amarillo (internos), verde (hojas)
B+	Internos: record con puertos ↓; Hojas: plaintext tabla HTML categoria+nombre	amarillo (internos), azul (hojas)

Los puertos en BTree y BPlus permiten que las flechas salgan desde el espacio exacto entre dos claves, comunicando visualmente el invariante: todo lo que hay bajo el puntero i contiene claves en el rango [keys[i-1], keys[i]).

Las hojas del BPlus tienen un puerto <next> con el simbolo → que representa el puntero next de la hoja. Las flechas punteadas entre hojas salen desde ese puerto y llegan al borde izquierdo de la siguiente hoja.

Justificación de t = 4

No se especifica el valor de t para los arboles B y B+. La elección de t = 4 se basa en el siguiente análisis para n = 1000 productos en memoria RAM:

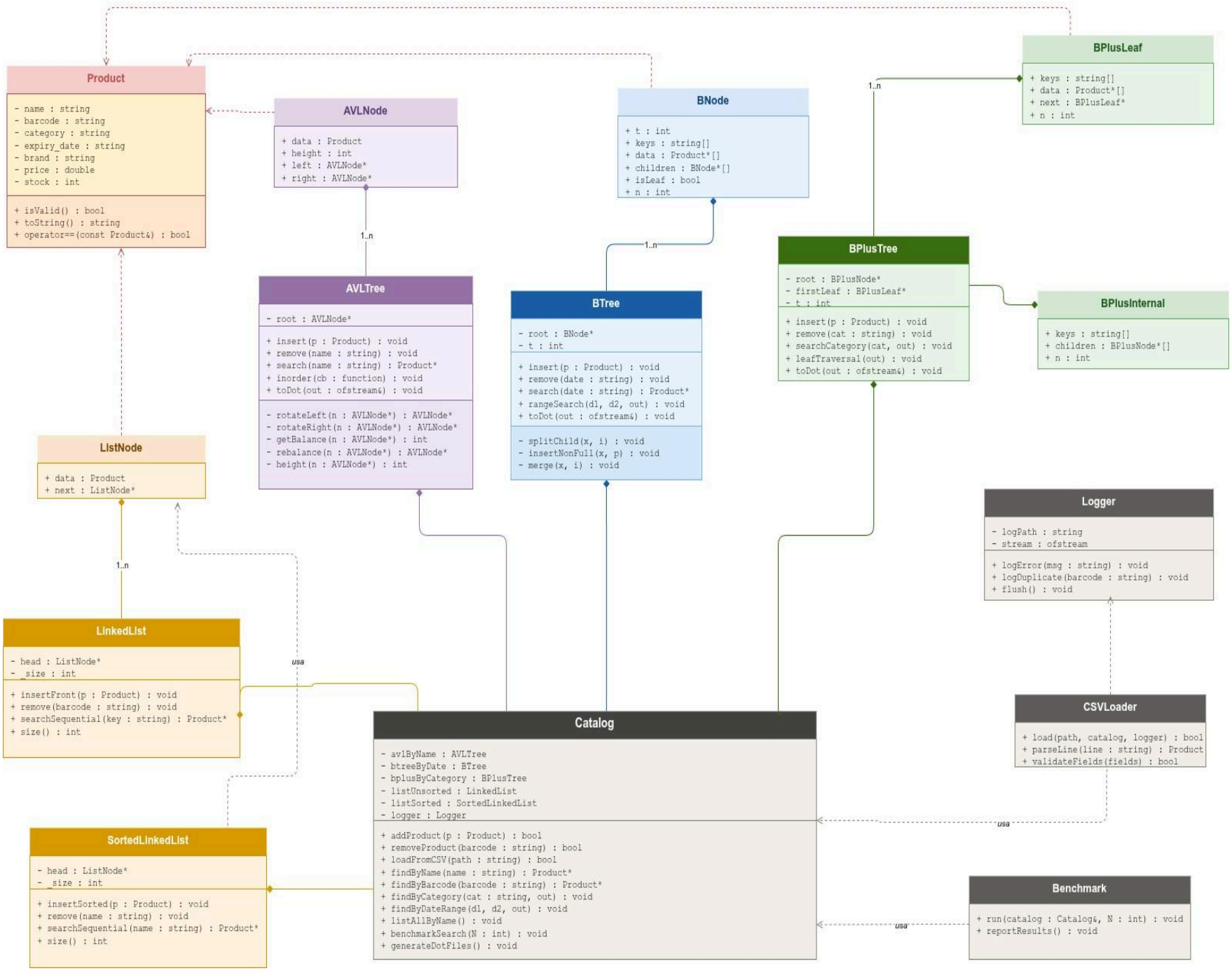
t	Claves min/max por nodo	Altura maxima (n=1000)	Keys en nodo lleno	Legibilidad en .dot
2	1 - 3	10	3	Alta

t	Claves min/max por nodo	Altura maxima (n=1000)	Keys en nodo lleno	Legibilidad en .dot
3	2 - 5	7	5	Alta
4	3 - 7	5	7	Alta (óptimo)
5	4 - 9	4	9	Media
10	9 - 19	3	19	Baja - saturado
20	19 - 39	2	39	Ilegible

Razonamiento para t = 4:

- De t=2 a t=3: la altura baja 3 niveles (de 10 a 7). Ganancia significativa.
- De t=3 a t=4: la altura baja 2 niveles más (de 7 a 5). Todavía ganancia relevante.
- De t=4 a t=5: la altura baja solo 1 nivel (de 5 a 4). Comienza la zona de rendimientos decrecientes.
- De t=5 en adelante: la altura cae menos de 1 nivel por cada incremento de t. La ganancia marginal es irrelevante en RAM.

t = 4 es el punto donde se maximiza la reducción de altura antes de entrar en la zona de rendimientos decrecientes, manteniendo los nodos con 7 claves, que es legible en los archivos .dot requeridos por el proyecto. El código es idéntico al de t=2 o t=3; solo cambia la constante BTREE_T.


Product
- name : string
- barcode : string
- category : string
- expiry_date : string
- brand : string
- price : double
- stock : int
+ isValid() : bool
+ toString() : string
+ operator==(const Product&) : bool
AVLNode
+ data : Product
+ height : int
+ left : AVLNode*
+ right : AVLNode*
BNode
+ t : int
+ keys : string[]
+ data : Product*[]
+ children : BNode*[]
+ isLeaf : bool
+ n : int
BPlusLeaf
+ keys : string[]
+ data : Product*[]
+ next : BPlusLeaf*
+ n : int
1..n
1..n
1..n
AVLTree
- root : AVLNode*
+ insert(p : Product) : void
+ remove(name : string) : void
+ search(name : string) : Product*
+ inorder(cb : function) : void
+ toDot(out : ofstream&) : void
- rotateLeft(n : AVLNode*) : AVLNode*
- rotateRight(n : AVLNode*) : AVLNode*
- getBalance(n : AVLNode*) : int
- rebalance(n : AVLNode*) : AVLNode*
- height(n : AVLNode*) : int
BTree
- root : BNode*
- t : int
+ insert(p : Product) : void
+ remove(date : string) : void
+ search(date : string) : Product*
+ rangeSearch(d1, d2, out) : void
+ toDot(out : ofstream&) : void
- splitChild(x, i) : void
- insertNonFull(x, p) : void
- merge(x, i) : void
BPlusTree
- root : BPlusNode*
- firstLeaf : BPlusLeaf*
- t : int
+ insert(p : Product) : void
+ remove(cat : string) : void
+ searchCategory(cat, out) : void
+ leafTraversal(out) : void
+ toDot(out : ofstream&) : void
BPlusInternal
+ keys : string[]
+ children : BPlusNode*[]
+ n : int
ListNode
+ data : Product
+ next : ListNode*
1..n
Logger
- logPath : string
- stream : ofstream
+ logError(msg : string) : void
+ logDuplicate(barcode : string) : void
+ flush() : void
LinkedList
- head : ListNode*
- _size : int
+ insertFront(p : Product) : void
+ remove(barcode : string) : void
+ searchSequential(key : string) : Product*
+ size() : int
usa
CSVLoader
+ load(path, catalog, logger) : bool
+ parseLine(line : string) : Product
+ validateFields(fields) : bool
Catalog
- avlByName : AVLTree
- btreeByDate : BTree
- bplusByCategory : BPlusTree
- listUnsorted : LinkedList
- listSorted : SortedLinkedList
- logger : Logger
+ addProduct(p : Product) : bool
+ removeProduct(barcode : string) : bool
+ loadFromCSV(path : string) : bool
+ findByName(name : string) : Product*
+ findByBarcode(barcode : string) : Product*
+ findByCategory(cat : string, out) : void
+ findByDateRange(d1, d2, out) : void
+ listAllByName() : void
+ benchmarkSearch(N : int) : void
+ generateDotFiles() : void
usa
SortedLinkedList
- head : ListNode*
- _size : int
+ insertSorted(p : Product) : void
+ remove(name : string) : void
+ searchSequential(name : string) : Product*
+ size() : int
Benchmark
+ run(catalog : Catalog&, N : int) : void
+ reportResults() : void
usa

TAD: LinkedList

Lista enlazada no ordenada — busqueda secuencial base

Variables de instancia (estructura interna)

```
head : ListNode* // puntero al primer nodo
_size : int // cantidad de nodos en la lista

struct ListNode {
data : Product // copia del producto
next : ListNode* // puntero al siguiente nodo
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	insertFront(p : Product) : void	p.isValid() == true	head apunta al nuevo nodo; _size++
Buscar	searchSequential(key : string) : Product*	key no vacia	retorna Product* o nullptr
Eliminar	remove(barcode : string) : void	lista no vacia	nodo eliminado; punteros reenlazados; _size--
Tamanio	size() : int	-	retorna _size

insertFront: O(1) | searchSequential: O(n) | remove: O(n) | size: O(1)

Justificacion: insertFront enlaza al frente sin recorrido O(1). searchSequential y remove recorren nodo a nodo hasta la coincidencia — peor caso n nodos.

TAD: SortedLinkedList

Lista enlazada ordenada por nombre — busqueda secuencial con early stop

Variables de instancia (estructura interna)

```
head : ListNode* // puntero al primer nodo (orden alfabetico por name)
_size : int // cantidad de nodos
struct ListNode { // misma struct que LinkedList, clase independiente
data : Product
next : ListNode*
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	insertSorted(p : Product) : void	p.isValid() == true	lista permanece ordenada por name
Buscar	searchSequential(name : string) : Product*	name no vacio	Product* o nullptr; se detiene si nodo.name > name
Eliminar	remove(name : string) : void	lista no vacia	nodo eliminado; orden conservado; _size--

insertSorted: O(n) | searchSequential: O(n) | remove: O(n) | size: O(1)

Justificacion: insertSorted recorre hasta hallar la posicion correcta. searchSequential puede detenerse antes (early stop), pero el peor caso sigue siendo O(n).

TAD: AVLTree

Arbol AVL ordenado por nombre del producto — busqueda binaria O(log n)

Variables de instancia (estructura interna)

```
root : AVLNode* // puntero a la raiz del arbol

struct AVLNode {
data : Product // producto almacenado
height : int // altura del nodo (para calcular balance)
left : AVLNode* // hijo izquierdo
right : AVLNode* // hijo derecho
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	insert(p : Product) : void	p.name no vacio; no duplicado	arbol balanceado; |balance| <= 1 en todo nodo
Buscar	search(name : string) : Product*	name no vacio	retorna Product* o nullptr
Eliminar	remove(name : string) : void	name existe en el arbol	nodo removido; arbol rebalanceado
Recorrer	inorder(cb : function) : void	arbol no vacio	cb llamado en orden alfabetico para cada nodo
Visualizar	toDot(out : ofstream&) : void	out abierto para escritura	archivo .dot valido escrito en out

insert: O(log n) | search: O(log n) | remove: O(log n) | inorder: O(n) | toDot: O(n)

Justificacion: la propiedad AVL garantiza h = O(log n). Cada operacion desciende por el arbol sin retroceder. El rebalanceo en el camino de retorno tambien es O(log n). inorder y toDot visitan todos los nodos exactamente una vez.

TAD: BTree

Arbol B (grado t) — clave: fecha de caducidad YYYY-MM-DD

Variables de instancia (estructura interna)

```
root : BNode* // puntero a la raiz
t : int // grado minimo (nodos tienen entre t-1 y 2t-1 claves)

struct BNode {
t : int // grado minimo
keys : string[] // claves (fechas YYYY-MM-DD), max 2t-1
data : Product*[] // punteros a productos, paralelo a keys
children : BNode*[] // punteros a hijos, max 2t
isLeaf : bool // true si es hoja
n : int // cantidad actual de claves
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	insert(p : Product) : void	p.expiry_date valido YYYY-MM-DD	clave insertada; splits propagados si nodo lleno
Buscar	search(date : string) : Product*	date formato YYYY-MM-DD	retorna Product* o nullptr
Rango	rangeSearch(d1, d2, out) : void	d1 <= d2; ambos formato valido	out contiene productos con fecha en [d1, d2]
Eliminar	remove(date : string) : void	date existe en el arbol	clave eliminada; merges o redistribucion aplicados
Visualizar	toDot(out : ofstream&) : void	out abierto para escritura	paginas con claves representadas en .dot

insert: O(log n) | search: O(log n) | remove: O(log n) | rangeSearch: O(log n + k) | toDot: O(n)

Justificacion: h = O(log_t n). Por pagina la busqueda lineal es O(t), constante respecto a n. rangeSearch: O(log n) para localizar d1, luego O(k) para recorrer los k elementos del rango.

TAD: BPlusTree

Arbol B+ (grado t) — clave: categoria del producto

Variables de instancia (estructura interna)

```
root : BPlusNode* // puntero a la raiz (interno u hoja)
firstLeaf : BPlusLeaf* // primera hoja (inicio del recorrido por nivel)
t : int // grado minimo

struct BPlusLeaf { // nodo hoja — almacena datos reales
keys : string[] // categorias, max 2t-1
data : Product*[] // punteros a productos, paralelo a keys
next : BPlusLeaf* // enlace al siguiente nodo hoja
n : int // cantidad actual de entradas
}

struct BPlusInternal { // nodo interno — solo claves guia
keys : string[] // claves separadoras, max 2t-1
children : BPlusNode*[] // punteros a hijos, max 2t
n : int // cantidad de claves
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	insert(p : Product) : void	p.category no vacio	dato en hoja; splits propagados hacia raiz si necesario
Buscar	searchCategory(cat, out) : void	cat no vacio	out con todos los productos de la categoria via hojas enlazadas
Eliminar	remove(cat : string) : void	cat existe en alguna hoja	dato eliminado de hoja; merge o redistribucion si underflow
Recorrer	leafTraversal(out) : void	arbol no vacio	out con todos los productos siguiendo next entre hojas
Visualizar	toDot(out : ofstream&) : void	out abierto para escritura	hojas enlazadas y nodos internos diferenciados en .dot

insert: O(log n) | remove: O(log n) | searchCategory: O(log n + k) | leafTraversal: O(n) | toDot: O(n)

Justificacion: datos solo en hojas; nodos internos son indices de navegacion. searchCategory desciende O(log n) y recorre hojas enlazadas O(k). leafTraversal ignora nodos internos gracias al puntero next.

TAD: Product

Variables de instancia

```
name : string // nombre del producto
barcode : string // codigo de barra (clave unica)
category : string // categoria (ej: lacteos, bebidas)
expiry_date : string // fecha de caducidad formato YYYY-MM-DD
brand : string // marca del producto
price : double // precio (debe ser > 0)
stock : int // unidades disponibles (debe ser >= 0)
```

Implementacion de metodos

```
isValid() : bool // true si name, barcode no vacios y price > 0 y stock >= 0
toString() : string // retorna string formateado con todos los campos
operator==(const Product&) : bool // true si barcode de ambos es igual
```

TAD: Menu

Variables de instancia

```
catalog : Catalog& // referencia al catalogo principal
running : bool // true mientras el menu principal siga activo
```

Implementacion de metodos

```
run() : void // bucle principal ejecutado hasta que el usuario elija salir
showMainMenu() : void //opciones numeradas impresas
readOption() : int // retorna entero leido
handleOption(opt : int) : void //accion correspondiente ejecutada sobre catalog
promptProduct() : Product //retorna Product con campos leidos de campo a campo
```

TAD: CSVLoader

Variables de instancia

```
Todos los metodos son estaticos. Reciben path, catalog y logger
como parametros en lugar de almacenarlos como atributos.
```

Implementacion de metodos

```
load(path, catalog, logger) : bool // productos validos insertados en catalog;
                                   //errores en logger;
                                   //retorna true si archivo abrio
parseLine(line : string) : Product // retorna Product con campos extraidos;
                                   //si malformada, retorna Product invalido
validateFields(fields : string[]) : bool //true si precio y stock son
                                  //numericos y fecha con formato correcto
```

TAD: Logger

Variables de instancia

```
logPath : string // ruta del archivo de log (ej: errors.log)
stream : ofstream // flujo de escritura abierto sobre logPath
count : int // cantidad de errores registrados en la sesion
```

Implementacion de metodos

```
logError(msg : string) : void // msg escrito en stream con timestamp; count++
logDuplicate(barcode : string) : void // entrada de duplicado escrita en
                                      stream con barcode y timestamp
flush() : void // buffer del stream vaciado
errorCount() : int // retorna count (errores registrados en la sesion)
```

TAD: Benchmark

Variables de instancia

```
catalog : Catalog& // referencia al catalogo a medir
N : int // numero de consultas por prueba (ej: 20)
M : int // numero de repeticiones para promediar (ej: 5)
results : string[] // resultados formateados para mostrar en consola
```

Implementacion de metodos

```
run() : void // N*M busquedas ejecutadas; tiempos almacenados en results
measureSearch(key, mode) : double // retorna tiempo promedio en microsegundos
                                  para N consultas
reportResults() : void // tabla comparativa impresa con tiempos
```

run: O(N*M * costo_busqueda) | measureSearch: O(N * costo_busqueda)

Justificacion: run ejecuta N*M busquedas; su costo depende del modo. measureSearch toma N muestras con chrono. reportResults recorre r resultados almacenados O(r) con r fijo y pequeno.

Metodologia de medicion

Se miden tres operaciones: búsqueda, inserción y eliminación. Cada operación se mide de forma **individual por estructura**, no de forma atómica a través de `Catalog::addProduct`, porque el objetivo es comparar el costo intrínseco de cada estructura, no el costo del sistema completo.

Se usa `std::chrono::high_resolution_clock`, que es el reloj de mayor resolución disponible en el sistema. El patrón es:

t0 es el tiempo antes de las N busquedas, **t1** después. La **diferencia** es el tiempo total para N búsquedas.

```
auto t0 = std::chrono::high_resolution_clock::now();
auto t1 = std::chrono::high_resolution_clock::now();
double us = std::chrono::duration<double, std::micro>(t1 - t0).count();
```

Por qué N=20 repeticiones y M=5 series.

<u>Permite comparar latencia por operación.</u>

Una sola medición no es confiable porque el sistema operativo puede interrumpir el proceso en cualquier momento para atender otras tareas. Para reducir ese problema se repite la medición varias veces.

Con **N=20** operaciones consecutivas por serie, el tiempo total de la serie se divide entre 20 para obtener el promedio por operación. Esto reduce el error de medición porque las interrupciones del SO afectan menos a un bloque de 20 operaciones que a una sola.

Con **M=5** series independientes se obtiene el promedio general, el mínimo y el máximo, que permiten ver la variabilidad de las mediciones. Si el mínimo y el máximo están muy separados, indica que hubo interferencia del SO en alguna serie.

El resultado final es:

avgTimeUs = total_tiempo / (M * N)	promedio por operación
minTimeUs = min_serie / N	mejor caso observado
maxTimeUs = max_serie / N	peor caso observado

Para medir **inserción** se insertan variantes con barcodes únicos directamente en cada estructura (no via Catalog::addProduct) y se eliminan inmediatamente después de medir. Esto evita dos problemas: que el catálogo crezca indefinidamente durante el benchmark, y que la unicidad de barcode interfiera con mediciones repetidas.

Para medir **eliminación** se toma el primer producto del catálogo como sujeto, se elimina de la estructura, se mide el tiempo, y se reinserta antes de la siguiente iteración. Esto garantiza que siempre hay un elemento que eliminar sin degradar el estado del catálogo.

Lógica completa de total_tiempo, min_serie y max_serie

Con N=20 y M=5 el flujo es este:

Serie 1: ejecutar 20 búsquedas seguidas -> medir tiempo total = t1
Serie 2: ejecutar 20 búsquedas seguidas -> medir tiempo total = t2
Serie 3: ejecutar 20 búsquedas seguidas -> medir tiempo total = t3
Serie 4: ejecutar 20 búsquedas seguidas -> medir tiempo total = t4
Serie 5: ejecutar 20 búsquedas seguidas -> medir tiempo total = t5

De esos cinco valores:

total_tiempo = t1 + t2 + t3 + t4 + t5 (suma de todas las series)
min_serie = min(t1, t2, t3, t4, t5) (la serie más rápida)
max_serie = max(t1, t2, t3, t4, t5) (la serie más lenta)

Luego para convertir a tiempo **por operación individual**:

avgTimeUs = total_tiempo / (M * N) = (t1+t2+t3+t4+t5) / (5 * 20) = suma / 100
minTimeUs = min_serie / N = min(t1..t5) / 20
maxTimeUs = max_serie / N = max(t1..t5) / 20

Generación de imágenes

data/

productos.csv	1000 Productos
productos_demo.csv	20 Productos
productos_errores.csv	

report/

productos_AVL.dot
productos_AVL.png
productos_BTree.dot
productos_BTree.png
productos_BPlus.dot
productos_BPlus.png

para ambas pruebas se generaron resultados

Resultados

Búsqueda por nombre

Se comparan LinkedList (búsqueda secuencial sin orden), SortedLinkedList (búsqueda secuencial con early stop) y AVLTree (búsqueda binaria). Los cuatro casos evalúan posiciones distintas dentro de cada estructura.

Este es el conjunto de datos principal. Con 1000 productos las diferencias teóricas se vuelven claramente observables.

Estructura	hit_random	miss	hit_first	hit_last
LinkedList	**3.7114**	**25.8514**	**9.9173**	**26.3561**
SortedList	**21.8090**	**51.0793**	**4.0898**	**6.7854**
AVL	6.9718	7.2881	7.0702	4.4263

Caso	Ganador	Explicacion
hit_random	LinkedList (3.7us)	El elemento buscado esta en las primeras posiciones de la lista no ordenada (insertFront)
miss	AVL (7.3us)	La lista recorre 1000 nodos; la SortedList recorre hasta superar la clave; el AVL solo baja log2(1000)=10 niveles
hit_first	SortedList (4.1us)	Early stop en el primer nodo; mejor caso absoluto para la lista ordenada
hit_last	AVL (4.4us)	La lista no ordenada recorre hasta el ultimo nodo (26us); el AVL no depende de la posicion

La SortedList en miss (51us) es el resultado más llamativo: es peor que LinkedList porque además de recorrer mas nodos, realiza comparaciones lexicográficas más costosas en cada nodo para decidir el early stop. En este catálogo el elemento buscado ('ZZZZZ') supera a todos los nombres existentes, por lo que el early stop nunca se activa y la lista recorre los 1000 nodos igualmente.

Inserción

La inserción se mide por estructura de forma independiente, no de forma atómica a través del Catálogo. Se inserta un producto con barcode único y se elimina inmediatamente después de cada medición para no alterar el estado del catálogo.

Estructura	Demo 20 prod (us)	Errores 5 prod (us)	1000 prod (us)
LinkedList	**2.5017**	**2.5125**	**2.4282**
SortedList	2.8384	3.5259	4.8997
AVL	4.5670	4.5315	4.4941
BTree	2.7861	2.9421	4.0610
BPlus	3.0263	3.2109	4.5275

LinkedList es consistentemente la más rápida porque insertFront es O(1) puro, sin búsqueda previa ni rebalanceo. SortedList crece de 2.8us a 4.9us al pasar de 20 a 1000 productos, lo que confirma su costo O(n) de búsqueda de posición. AVL se mantiene estable (~4.5us) en los tres tamaños, lo esperado para O(log n). BTree y BPlus crecen moderadamente con el tamaño del catálogo.

Eliminación

La eliminación se mide tomando el primer producto de la lista como sujeto. Después de cada eliminación el producto se reinserta para mantener el catálogo constante.

Estructura	Demo 20 prod (us)	Errores 5 prod (us)	1000 prod (us)
LinkedList	2.1840	2.1460	3.4405
SortedList	**2.4647**	**2.1942**	**28.7019**
AVL	3.7952	3.6677	4.8215
BTree	3.4761	3.7257	4.7943
BPlus	3.8239	3.5553	4.1324

El resultado más relevante es SortedList en 1000 productos (28.7us): la eliminación requiere encontrar el nodo primero (busqueda O(n)) antes de poder re-enlazarlo. Con 1000 productos esa búsqueda previa domina el tiempo total. El valor 28us es coherente con el miss de 51us: la eliminación hace la mitad del trabajo (el producto existe, el early stop si actúa antes). LinkedList en eliminacion (3.4us con 1000 productos) es mas costosa que en demo (2.2us) porque el sujeto de prueba es el primer nodo de la lista, pero después de cada reinsercion con insertFront, el orden cambia y en siguientes iteraciones puede no estar al frente.

Conclusiones

Operación / Estructura	Big-O teorico	Observado (1000 prod)	Coincide
Búsqueda LinkedList hit_last	O(n)	26.3 us	Si - peor caso claro
Búsqueda SortedList miss	O(n)	51.1 us	Si - recorre todo sin early stop
Búsqueda SortedList hit_first	O(1)	4.1 us	Si - primer nodo inmediato
Búsqueda AVL (todos los casos)	O(log n)	4.4-7.3us	Si - constante independiente de caso
Inserción LinkedList	O(1)	2.4 us	Si - mas rapido y estable
Inserción SortedList	O(n)	4.9 us	Parcial - n=1000 aun no es grande

Operación / Estructura	Big-O teorico	Observado (1000 prod)	Coincide
Eliminación SortedList	O(n)	28.7 us	Si - búsqueda previa domina
Insercion/Eliminacion AVL	O(log n)	4.5-4.8us	Si - estable entre tamaños

Sobre la búsqueda

El árbol AVL es la única estructura cuyo tiempo de búsqueda no depende de la posición del elemento ni del tamaño del catálogo de forma lineal. Con 1000 productos sus cuatro casos están entre 4.4us y 7.3us, mientras que LinkedList varía entre 3.7us y 26.4us dependiendo de si el elemento está al inicio o al final de la lista.

La SortedLinkedList tiene la ventaja del early stop, que la hace ganar en hit_first (4.1us, el mejor resultado de todo el benchmark de búsqueda) pero la perjudica en miss cuando el catalogo es grande, porque la clave inexistente 'ZZZZZ' supera todos los nombres y obliga a recorrer la lista completa antes de concluir que no existe.

Sobre la inserción

LinkedList con insertFront es el método de inserción más rápido (2.4us) porque no requiere ninguna búsqueda previa. Es el costo mínimo posible para insertar en una estructura dinámica. Todos los demás métodos pagan un costo adicional por mantener un invariante: orden alfabético en SortedList, balance en AVL, o estructura de páginas en BTree y BPlus.

El costo de inserción en AVL (~4.5us) es estable entre los tres tamaños de catálogo, lo que confirma que O(log n) escala bien. BTree y BPlus tienen costos similares al AVL, lo que es esperado porque todos son estructuras O(log n).

Sobre la eliminación

La eliminación en SortedList con 1000 productos (28.7us) es el caso más dramático del benchmark. El costo proviene de la búsqueda previa O(n) que es obligatoria antes de poder re-enlazar los punteros. Esto ilustra el costo oculto de mantener una lista ordenada: la inserción y eliminación son O(n) aunque el re-enlace en si sea O(1).

Las estructuras de árbol (AVL, BTree, BPlus) tienen costos de eliminación similares entre sí (4.1us a 4.8us) porque todas mantienen O(log n) gracias a sus mecanismos de rebalanceo o merge.